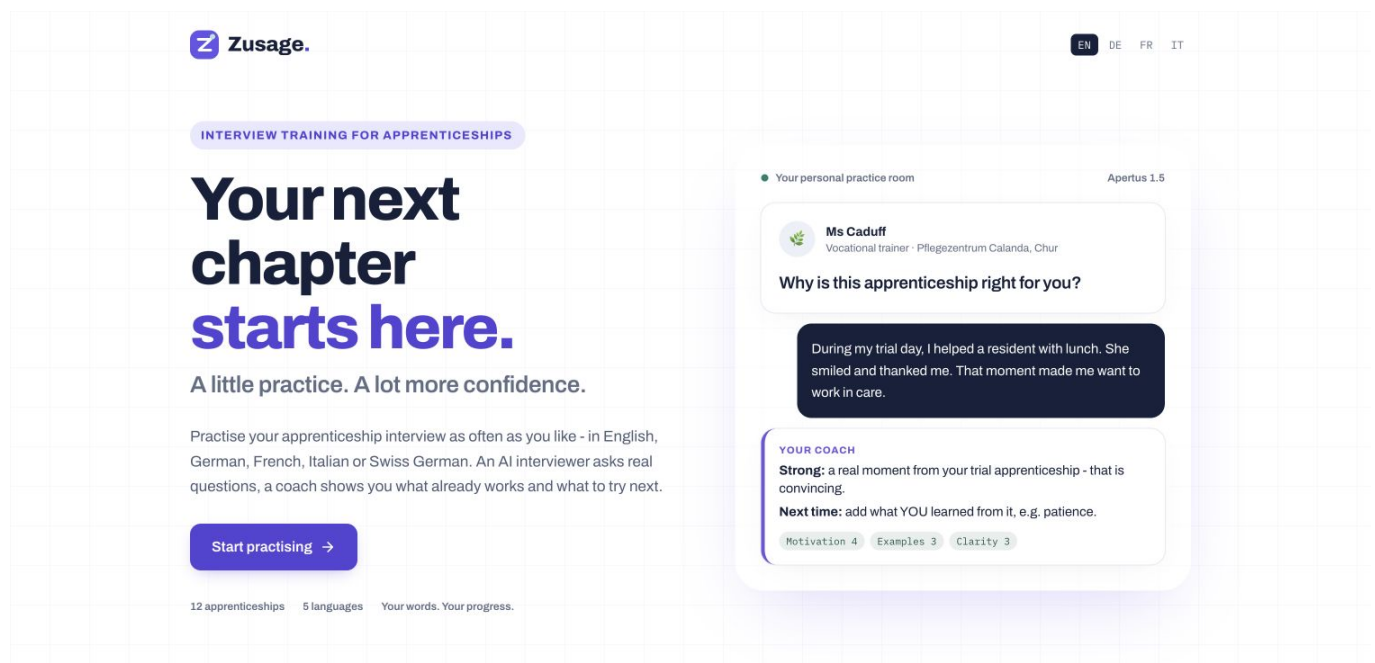


PURPOSE AND PRODUCT

A fair chance to practise.

An interview coach for young people entering Swiss apprenticeships. Built around their own words, practical feedback and the confidence to try again.

The first real interview should not be a student's first rehearsal. Teachers cannot repeat individual interview practice for every learner. Zusage offers a safe practice room and a useful class view, while keeping personal answer text under student control.



From practice to a real application

Choose from 12 apprenticeships, three interviewer styles and five interview languages. Answer, self-assess, inspect verified evidence, retry and receive a final report. Weak answers return as short spaced drills. Applications, outreach, prep packs and follow-up reminders keep that practice relevant.

Required languages, plus two

German, French and Italian meet the challenge requirement. English opens by default; Swiss German is available as a beta interview language. The saved app preference and interview language are independent.

ARCHITECTURE

Small model. Clear structure.

Apertus handles language and adaptation. Deterministic code handles interview flow, evidence validation, persistence and learning schedules.

React interface

Vite, TypeScript and self-hosted fonts. Desktop sidebar, mobile navigation, responsive tracker and an accessible dialog system.

FastAPI and LangGraph

A checkpointed interview graph drives introduction, motivation, strengths, situations and candidate questions. Aegra-compatible deployment and an HTTP benchmark harness are included.

Apertus 1.5 8B

One structured response combines assessment and reaction for each answer. Separate bounded calls tailor a job advert and synthesize the final report. Personal keys use the same configured Apertus endpoint and model.

Validation and persistence

Quotes must occur in the actual answer. Scores and output shapes are checked. SQLite supports a school server; PostgreSQL supports the hosted deployment.

Firebase Hosting serves static assets and forwards API routes to Cloud Run. Cloud SQL persists accounts and practice data. Secret Manager stores server credentials; the personal key is encrypted with a key derived from the stable session secret.

LEARNING DESIGN

Reflect. Retry. Remember.

Development-oriented feedback is useful only when the student understands the next action and can put it into practice.

Six criteria, with evidence

Clarity, relevance, motivation, self-awareness, communication and concrete examples use a 1-4 scale. Feedback anchors to validated answer quotes. Invalid or absent evidence is not presented as a fabricated citation. Malformed model results trigger a clearly labelled fallback.

A student stays in control

Training asks students to assess their own answer before seeing the coach. Immediate retries compare attempts. Dress rehearsal withholds coaching until the end. A confidence check before and after practice records self-perception without treating it as an objective learning result.

A practice loop, not a score chase

Weak answers become Leitner-style drill cards with expanding intervals. The report highlights strengths and concrete goals. A real story bank supports truthful examples instead of memorized model answers. Momentum rewards real actions, with a weekly goal and activation checklist.

Offerloop workflows, adapted

Application tracking, a replayable tour, posting extraction, six draft/prep types, CSV ingestion, funnel analytics and scheduled follow-ups support the same learner journey. Drafts use application context, the profile and past writing. They remain editable and require human review before sending. Role profiles, example answers and evaluation scenarios are original synthetic teaching data; no human-subject dataset was collected.

Voice ready

The interviewer can read questions using browser text-to-speech. Dictation is optional and labelled because some browser vendors process audio remotely. Typed practice always remains available.

PRIVACY AND RELIABILITY

Private by design.

Minors need useful practice without unnecessary collection or unrestricted access to their answers.

Minimal accounts and explicit sharing

Students sign up with a username and password, with no email or address required. Teacher access is restricted to their class. Scores can be shared for class support; answer text needs explicit student consent. Leaving or switching a class revokes sharing.

Credentials and account rights

Argon2 protects passwords. HttpOnly, Secure and SameSite cookies carry signed sessions. Password changes invalidate older sessions. Recovery codes are hashed, single-use and displayed only when generated. Export omits secret keys and recovery hashes; deletion removes the account and its owned rows.

Private demo workspaces

Each student or teacher demo receives a separate copy of the fictional class. Visitors do not overwrite one shared account. Ownership checks protect applications, interviews and drafts. PostgreSQL row locks protect interview updates.

Safety, retention and honest hosting

Contact identifiers are redacted before interview content reaches Apertus. Distress pauses an interview and requires explicit resume. Old transcript text and quoted report content are removed after the retention period during maintenance; numeric progress remains. The hourly task also runs retention maintenance.

The hosted demo is not an on-premise claim

The demo uses Google Cloud and the organiser's inference endpoint. Schools can deploy the same application and Apertus locally. Legal compliance and any production school rollout still require an operator review, appropriate policies and agreements.

VALIDATION AND JUDGING

Evidence, with limits.

Functional reliability tests and a judge benchmark answer different questions. This report does not claim an official benchmark win.

Current automated suite

77 backend tests cover API ownership, authentication, teacher consent, redaction, safety, malformed provider responses, retry handling, retention, typed profiles, imports, draft isolation, recovery, encryption, scheduler authentication and analytics. Nine frontend tests cover formatting, default language, saved preference and translated core screens. TypeScript production builds and Ruff pass.

Five-language live HTTP run

Language	Answers	Model calls
German	7	8
French	8	9
Italian	7	8
English	8	9
Swiss German	8	9

The run completed 38 answers with 43 model calls: 1.13 calls per answer, below the required average of five. No offline fallback occurred in that run. It checked completion, reports, real-mode feedback gating, export, closed-session rejection, session isolation and teacher consent. These are observed run results, not a guarantee under every provider condition.

Required gates and remaining evaluation

Cloud Build built the Docker image and Cloud Run executed it. A real local 4-bit Apertus run on Apple M4 Pro (24 GiB unified memory) completed 8 answers with 9 calls, no fallbacks and 5.155 GiB peak Metal allocation in 87.13 seconds. This demonstrates consumer Apple GPU execution below the memory/call limits. Metal shared memory is not dedicated NVIDIA VRAM; NVIDIA Docker inference remains unmeasured.

Judging weights are performance 50%, consistency 25% and innovation 25%. Historical exploratory evaluations in the repository are not current official judge results. The local measurement uses the supplied scripted answer bank and greedy sampling (temperature 0); benchmark code is in commit c01988d. Use the official judge prompts once provided.

DELIVERY AND NEXT STEPS

Ready to try. Ready to inspect.

A clean web URL, reproducible source and practical testing instructions keep the project reviewable.

Try the working product

Open <https://zusage.web.app>. Choose Try as student, follow the tutorial, select an interview language and practise. Retry a short answer with a real example, inspect the report and progress, then explore applications, writing, imports and reminders. Sign out and choose Try as teacher to inspect consent-aware classroom support.

Reproduce and deploy

From the repository root, copy `track_2a/.env.example` to `track_2a/.env`, configure the Apertus key and run `make run`. Use `make test` and `make lint` for validation. In `track_2a`, `scripts/deploy.sh` builds, deploys Firebase and Cloud Run, waits for routing and runs a live interview selfcheck. Dedicated resources and Secret Manager entries must already exist.

What works in the hosted version

The API uses persistent Cloud SQL, isolated demos, encrypted optional personal Apertus keys and an authenticated hourly reminder scan. Browser notifications are opt-in. Drafts never send automatically. Public HTTPS job retrieval may be blocked by a posting site; direct advert text is supported.

What comes next

Run the official judge benchmark, verify the NVIDIA Docker inference path, and work with vocational educators on feedback quality and dialect evaluation. Assess accessibility with assistive technology users. Add Profolio integration and school-level deployment governance before a real student pilot.

Submission review

The revised demo and captions were approved by the participant. The submission uses the two-minute cut, the default aperture-redesign branch and this six-page report. Code is Apache-2.0, original non-code output CC-BY-4.0 and original datasets CDLA-Permissive-2.0; third-party licenses are preserved.

Source: <https://github.com/shi1720/zusage> Challenge: <https://hackapertus.notion.site/track-2a-fhgr> Live product: <https://zusage.web.app>